

Full Stack Developer Task: Project Management System (Web + Mobile)

Build a web application and a mobile app that let users manage projects and tasks. Both apps use the same backend and database, so a user can log in on either one and see and manage the same projects and tasks.

Users should be able to create projects, organize tasks within projects, track progress, and view overall statistics through a dashboard.

Functional Requirements

1. User Authentication

The application must support:

- User Registration
- User Login
- User Logout

User Fields

- Full Name
- Email Address
- Password

Requirements

- Email addresses must be unique.
- Passwords must never be stored in plain text.
- Authenticated users should remain logged in until logout or token expiration.
- One account works on both web and mobile: a user who registers on one can log in on the other.

2. Project Management

Users should be able to:

- Create a project
- View project details
- Edit a project
- Delete a project

- View all projects they own

Project Fields

- Project Name
- Description
- Status: Not Started, In Progress, Completed
- Start Date
- End Date
- Created Date

3. Task Management

Each project can contain multiple tasks. Users should be able to:

- Create tasks
- Edit tasks
- Delete tasks
- Mark tasks as completed
- View tasks under a project

Task Fields

- Task Name
- Description
- Priority: Low, Medium, High
- Status: Pending, In Progress, Completed
- Due Date
- Created Date

4. Dashboard

Provide a dashboard displaying:

- Total Projects
- Total Tasks
- Completed Tasks
- Pending Tasks
- Projects In Progress

The dashboard should update based on the authenticated user's data.

5. Search and Filtering

Users should be able to:

- Search projects by name
- Search tasks by name
- Filter projects by status
- Filter tasks by status
- Filter tasks by priority

6. Mobile App

Build a mobile app that talks to the same backend and database as the web application. There must be no separate backend for mobile.

On the mobile app, users should be able to:

- Register, log in and log out with the same account they use on the web
- View the dashboard
- View all their projects and the tasks under each project
- Create, edit and delete tasks
- Mark tasks as completed and change task status and priority
- Search tasks and filter them by status and priority

Requirements

- Android is required; iOS is optional.
- A change made on one platform appears on the other after a refresh (pull-to-refresh on mobile).
- The login token is stored in secure device storage (Android Keystore / iOS Keychain), not in plain local storage.
- When the token expires, the user is sent back to the login screen with a clear message.
- No network: show a clear message instead of a crash or blank screen.

Technical Requirements

Frontend (Web)

Choose one: React or Next.js.

- Responsive Design
- Proper Component Structure

- Form Validation
- Loading Indicators
- Error Handling
- Clean User Experience

Mobile App

Choose one: React Native (Expo or bare) or Flutter.

- Proper Navigation and Screen Structure
- Form Validation
- Loading Indicators and Pull-to-Refresh
- Error Handling (including no network and expired login)
- Secure Token Storage
- Clean User Experience on a phone screen

Backend

Choose one: Node.js with Express, or NestJS. One backend serves both the web and the mobile app.

- REST API Architecture
- Proper Route Organization
- Middleware Usage
- Error Handling
- Logging
- Clean Code Structure
- CORS configured for the web app's domain

Database

Choose one: PostgreSQL or MySQL.

- Proper Relational Design
- Foreign Key Relationships
- Normalized Database Structure

Security Requirements

Security is an important part of this assessment.

Authentication Security

- Passwords must be hashed using bcrypt or equivalent.
- Plain-text passwords must never be stored.
- Protected APIs must require authentication.

Authorization

Users must only be able to view, modify and delete their own projects and tasks, on both web and mobile. Users must not be able to access data belonging to other users.

Input Validation

Validate all incoming requests on the backend, whichever app sent them. Examples: required fields, invalid email formats, invalid date values, empty strings, invalid enum values. Return appropriate error responses.

API Security

- JWT Authentication
- Authentication Middleware
- Protected Routes

Sensitive information must not be exposed in API responses.

Database Security

The application must be protected against SQL injection. Use ORM methods, query parameterization or prepared statements. Avoid raw user input in database queries.

Basic Rate Limiting

Implement rate limiting on authentication endpoints to reduce brute-force risk, for example by limiting repeated login attempts from the same IP.

API Expectations

The following endpoints are expected as a minimum. The web app and the mobile app must both use them.

Authentication

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET  /api/auth/me
```

Projects

```
GET    /api/projects
GET    /api/projects/{id}
POST   /api/projects
PUT    /api/projects/{id}
DELETE /api/projects/{id}
```

Tasks

```
GET    /api/tasks
GET    /api/tasks/{id}
POST   /api/tasks
PUT    /api/tasks/{id}
DELETE /api/tasks/{id}
```

Dashboard

```
GET /api/dashboard
```

Documentation Requirements

Provide:

- Project Setup Instructions (backend, web and mobile)
- Environment Variable Documentation
- Database Setup Instructions
- API Documentation
- How to run the mobile app against your deployed backend

The application should be easy to run by another developer.

Submission Requirements

Submit the following:

1. Public GitHub Repository Link, viewable without logging in (one repository, or separate ones for backend, web and mobile)

2. Database Schema or ER Diagram
3. API Documentation
4. README File
5. Deployment URL for the web app and the backend
6. Android APK, or an Expo / Firebase App Distribution link, for the mobile app
7. A 5-minute screen recording: log in with the same account on web and mobile, create a task on one, and show it on the other

Bonus Features (Optional)

The following features are optional and will be considered positively during evaluation:

- Docker Support
- Unit Tests
- Integration Tests
- Pagination
- Sorting
- Audit Logs
- Role-Based Access Control
- CI/CD Pipeline
- Refresh Tokens
- Push Notifications for tasks due tomorrow
- Offline viewing of tasks on mobile
- Shared types or validation between web, mobile and backend

Important Notes

- You may use libraries, frameworks, AI-assisted tools and online resources.
- You should be able to explain your implementation and design decisions during the review session.
- Code readability, maintainability and security practices are important evaluation criteria.
- The application does not need to be production-ready but should demonstrate good engineering practices.
- Use only test data. Do not use real personal data.